

claude code 分享 轨道0

Claude Code Agent 全景概览——从原理到应用，读懂智能 Agent 的核心逻辑与价值

分享人：王鑫、胡凯惟 | 轨道 0 — Agent 原理概览

时间：2026.4.9 周四 | 时长：约 26 分钟（Part0: 5min + Part1: 15min + Part4: 6min）

Part 0：开篇引入

- 背景铺垫：AI Agent的发展现状
- 分享预告：明确本次概览的核心目标——帮大家快速掌握Agent核心逻辑、组成及实践价值

一、背景铺垫：AI Agent发展现状

1. 从「会说话」到「会办事」

大模型普及后，大家已经习惯用对话完成写作、问答、简单分析。下一阶段的共识是：让 AI 在明确目标下，多步执行、调用工具、对接系统——这就是业界常说的 **Agent（智能体）** 方向。企业侧也在从「试点聊天机器人」转向「可度量、可运维的智能体工作流」。

2. 行业侧的两个信号

- 采用面在扩大：多家行业研究与报告显示，企业正在规划或落地各类「自主/半自主」AI 能力；同时，真正规模化落地的比例仍有限，大量团队卡在集成、安全、可观测性与责任边界上。
- 形态在分化：既有面向办公与业务流程的通用助手型 Agent，也有深度嵌入研发环节的编码 Agent

那么今天我们要分享什么：无论是产品、研发、安全还是效能团队，接下来都会 increasingly 遇到同一套问题：Agent 能做什么、不能做什么、如何治理、如何与现有工具链结合。

今天对于Claude Code，我们先做一张「地图」，后面各轨道再「按图施工」。

二、分享核心目标

- 是什么：用非技术听众也能听懂的方式，定义 Agent。
- 怎么转起来：用「闭环」讲清原理，不展开算法与工程细节。
- 价值在哪：对应到「感知—思考—行动—反馈」，并点名与后续分享的挂钩（主循环、工具调用等）。

- **怎么用：**典型场景、使用原则、常见坑。

本场（轨道 0）：全景 + 原理 + 应用观，帮助大家对齐概念与语言。

Agent 是理解产品行为与扩展方式的主线——我们先建立共同认知，再谈 Hooks、子 Agent、规范与落地案例；后续集团内会有针对 Claude Code 核心组件的一系列相关分享。

接下来我先从『Agent 是什么』讲起，把运转逻辑讲透，给大家后面听架构拆解和产品对比打基础。

Part 1：Agent核心原理解析

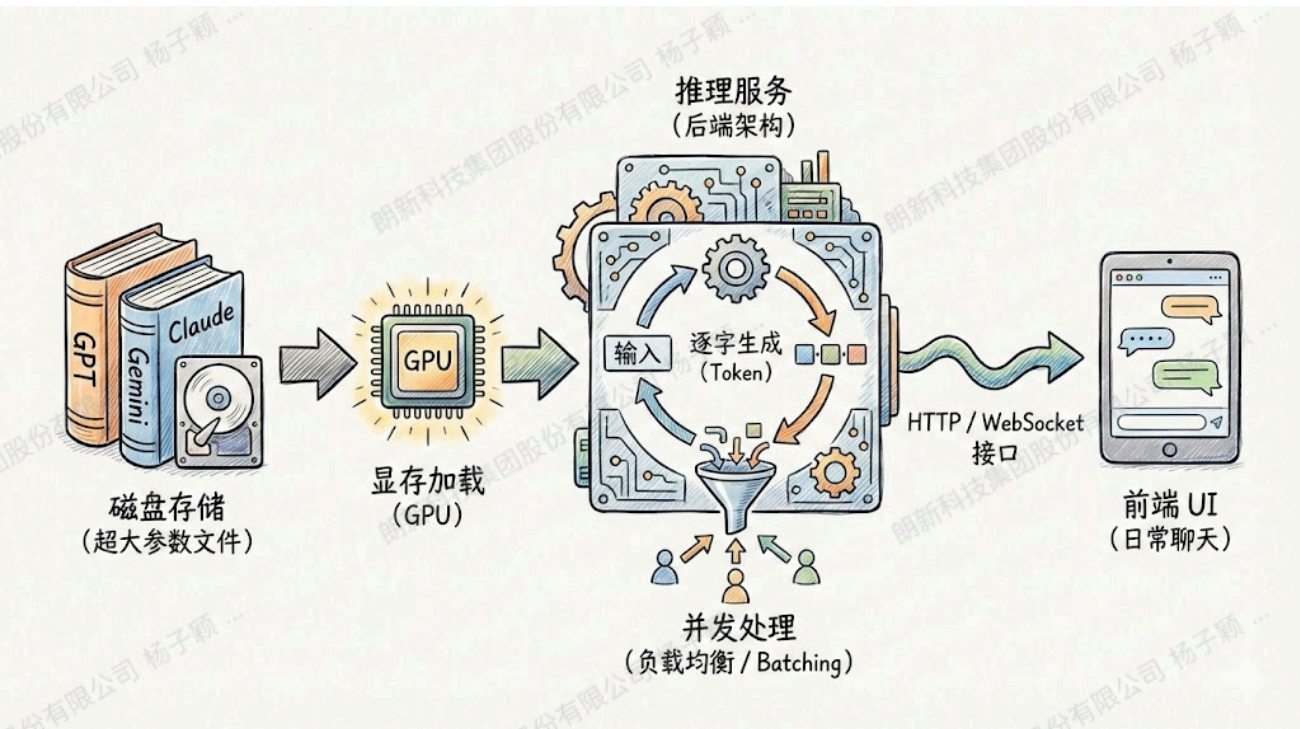
Agent 是一种在目标驱动下，能够持续与环境交互的系统：它接收信息与约束，进行推理与规划，选择并执行动作（包括调用工具），根据结果调整策略，直到任务完成或需要人类介入。

对比	说明
vs 单次问答	不是「一问一答就结束」，而是 多步推进 ，中间可能查资料、调接口、改文
vs 固定脚本	不是写死 if-else，而是 根据上下文与中间结果 决定下一步（在一定边界内「
vs 纯聊天	强调对 外部世界可执行 ：运行命令、读写资源、触发工作流等（视权限与工

一、基本拆解

1. 基础前提：模型推理服务

GPT、DeepSeek 等大模型，本质是存储在磁盘上的超大参数文件（通常高达几十 GB 甚至上百 GB）。



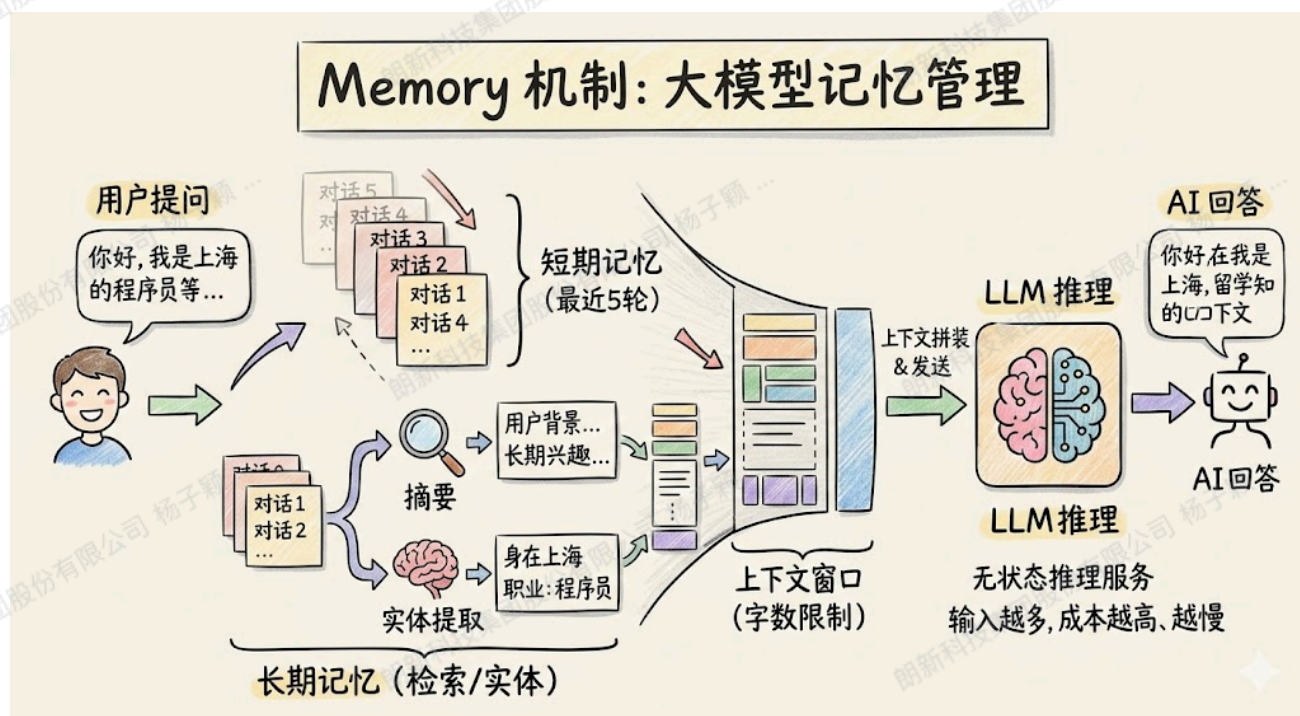
想要让大模型工作，需要一套专门的后端架构将其加载到显存中，对外提供 HTTP 或 WebSocket 接口。

这个接收用户请求、进行数学矩阵运算并逐字生成回复的服务，就是推理服务。

2. 对话记忆：Memory机制

为什么需要记忆：

- 推理服务本身是无状态的 HTTP 服务，请求处理完成后不会保留任何数据，不同请求可能被不同实例处理。
- 大模型每次处理信息都有字数限制（即上下文窗口 Context Window），且输入的字数越多，计算成本越高、响应越慢。因此我们也不能简单粗暴地把所有历史对话都塞进去，而是需要分层、按需管理：



记忆分类：

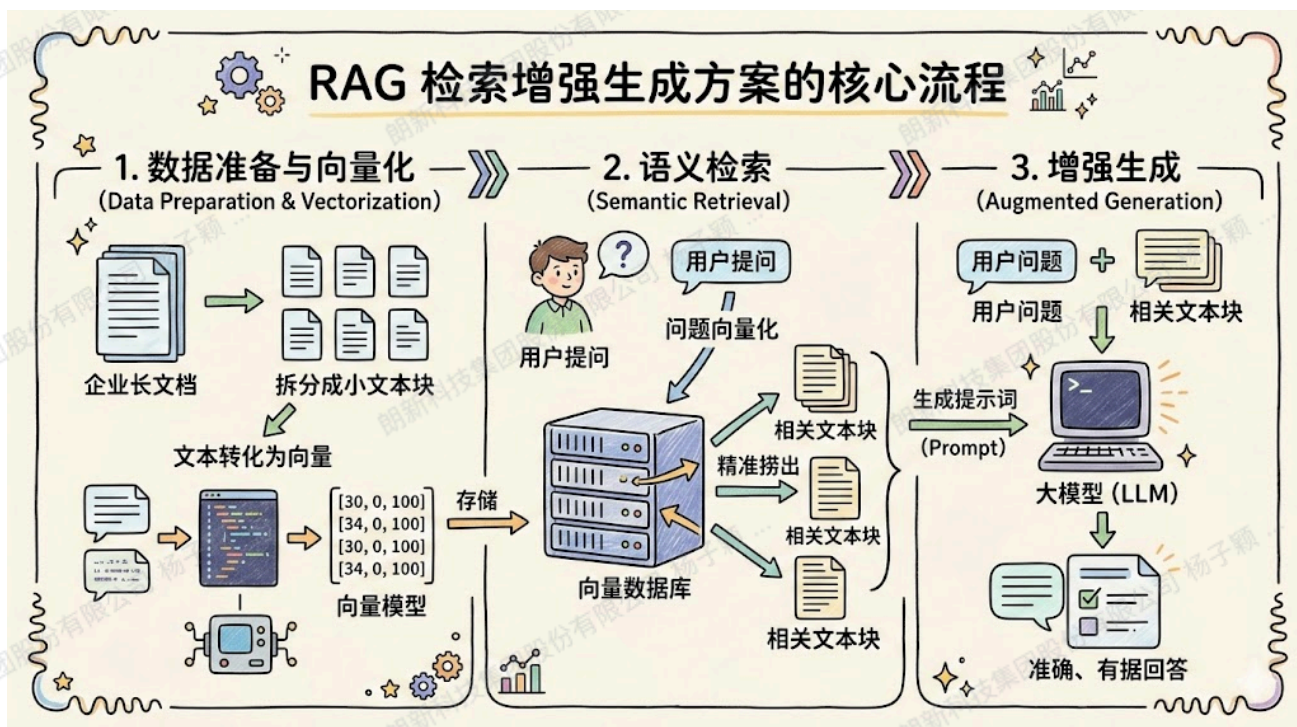
- 短期记忆：原封不动地保留最近几轮的对话原文，确保当前对话的连贯性。
- 长期记忆：对于久远的对话，系统会在后台触发另一个小模型，将冗长的历史对话压缩成简短摘要；或者提取出结构化的“实体特征”（例如：记住“用户是一名身在上海的程序员”），并存储在数据库中。

每次用户发起新提问时，系统会先提取并拼装这些记忆片段，再和当前问题一起发送给大模型。这套管理对话上下文、让AI具备“记忆能力”的机制，就是 **Memory**。

3. 外部知识：RAG检索增强

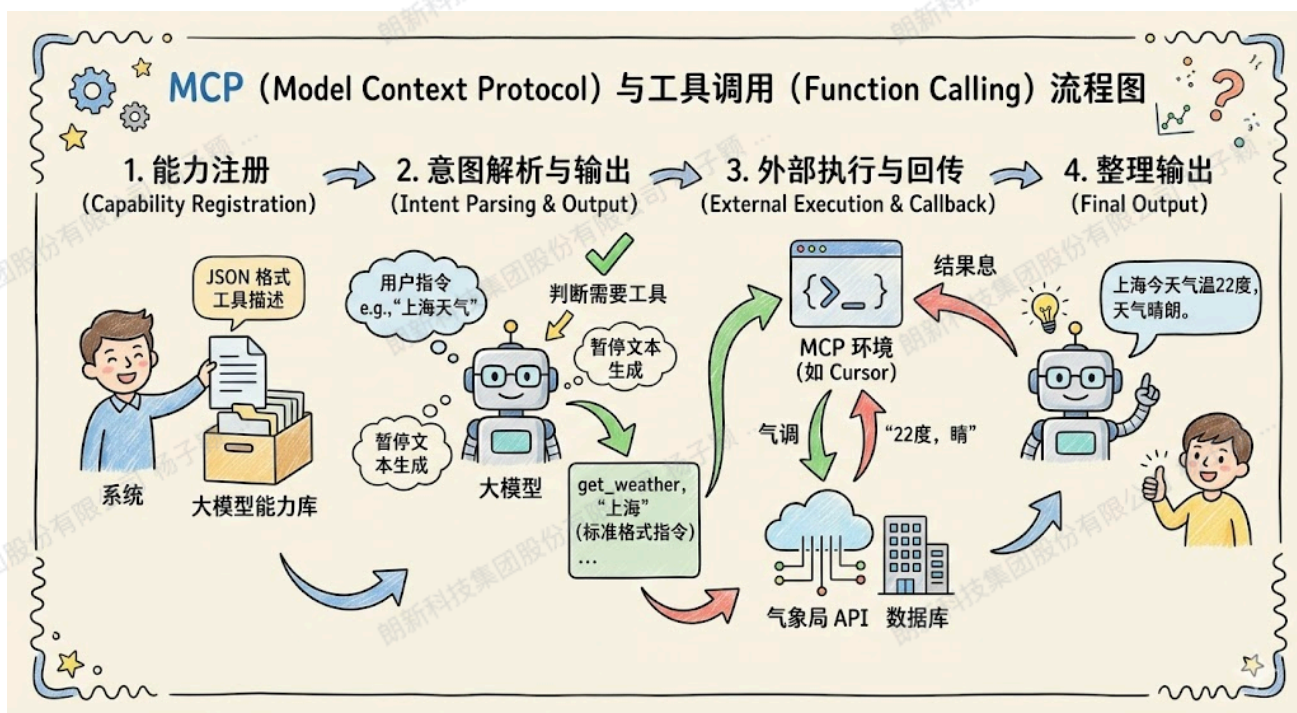
大模型的知识完全受限于它的训练数据。一旦训练完成，它的知识就“冻结”了，无法回答实时新闻或企业内部的保密文档，且容易一本正经地胡说八道。

RAG 是目前解决此问题最成熟的技术路径，其核心是“先查资料，再作答”：



4. 工具调用：MCP

有了 Memory 和 RAG，大模型具备了“内部记忆”和“外部知识”，但它依然是一个封闭在服务器里的纯文本处理系统，无法主动执行获取实时天气、读写本地文件、查询数据库等外部操作。

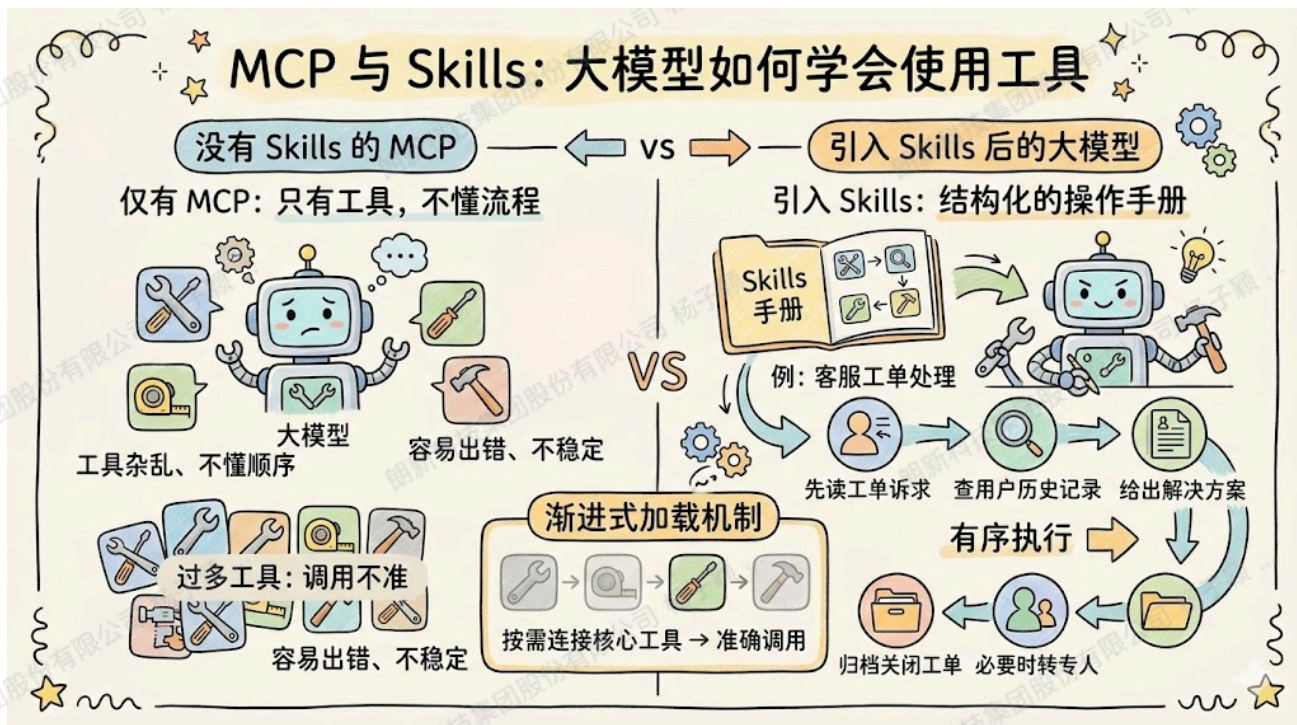


为了打破这种封闭，行业内发展出了工具调用能力，而 MCP 则是一套标准化、开源的工程协议规范，专门用于统一模型与外部工具的对接标准。

5. 流程规划：Skills能力

MCP 解决了“能不能调用工具”的问题，但大模型不知道何时用、按什么顺序用、如何组合工具，就像拥有工具却不懂修车流程。

MCP 与 Skills: 大模型如何学会使用工具



Skills 是一套结构化的操作手册/执行流程:

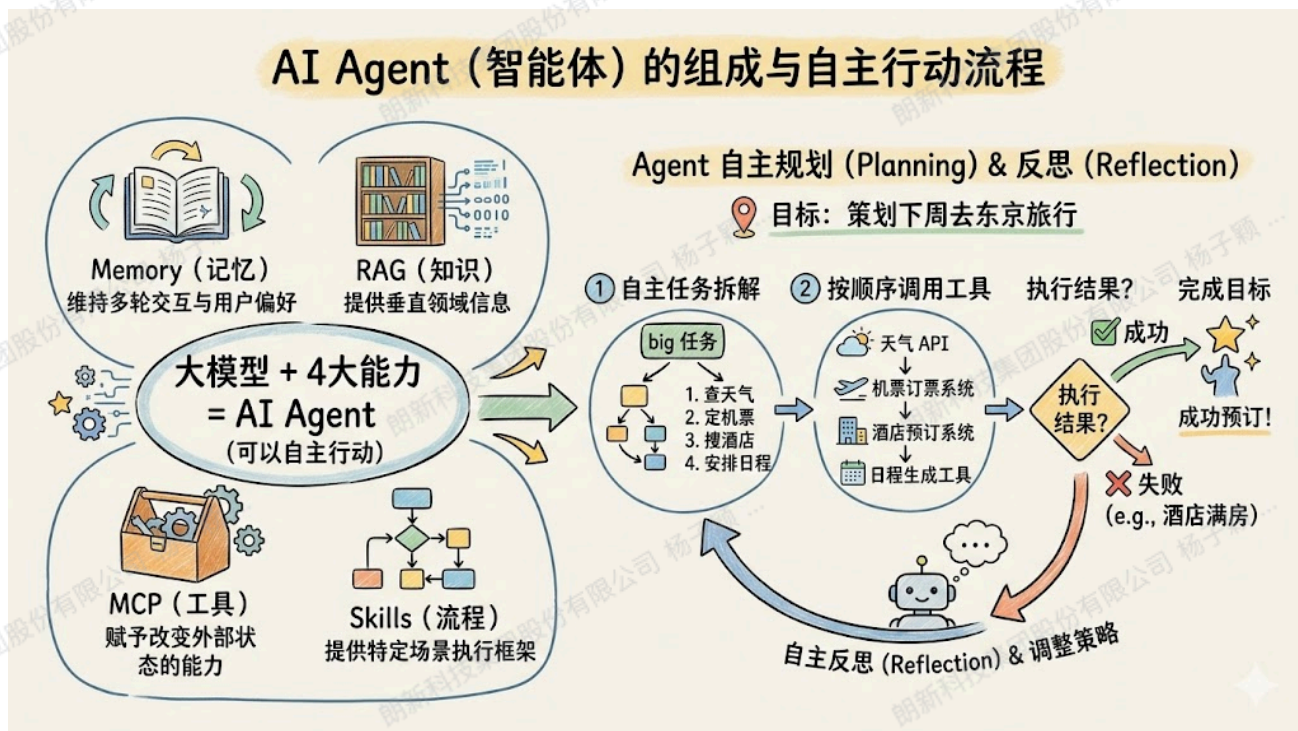
它明确规定了特定场景下, 工具的使用顺序、执行逻辑、注意事项。

例: 在客服工单处理场景中, MCP 提供查用户档案、查业务记录、发送话术、转人工、关闭工单等工具; 而 Skills 则定义「先读工单诉求→查用户历史记录→给出解决方案→必要时转专人→归档关闭工单」的完整流程。

同时, 依靠 Skills 核心的渐进式加载的机制, 可以也可以解决同时连接工具过多而调用不准确、不稳定的问题。

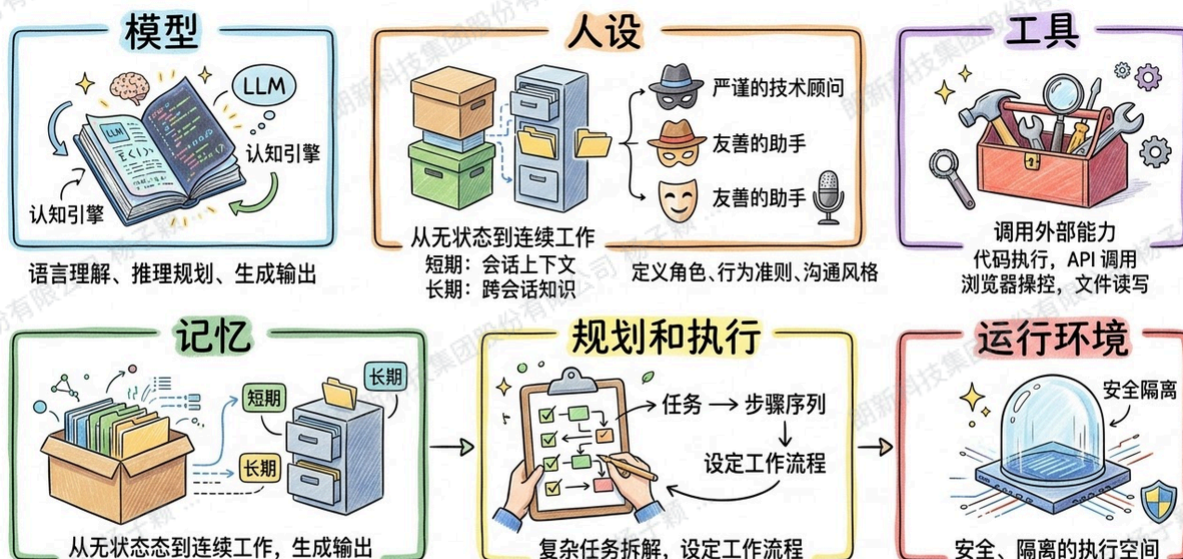
6. 完整的智能体: AI Agent

AI Agent (智能体) 的组成与自主行动流程



当一个大模型同时拥有以下能力后，就形成了可以 **自主行动、完成目标** 的 AI 系统，即 **AI Agent（智能体）**：

- Memory（记忆）：维持多轮交互的状态和用户偏好。
- RAG（知识）：提供执行任务所需的垂直领域信息。
- MCP（工具）：赋予系统改变外部状态的操作能力。
- Skills（流程）：提供特定场景下的执行框架。



Agent 核心要素

也就是说：

1. 感知（输入）

用户目标、任务约束、环境状态、工具返回、策略与规范（例如项目内的编码规范说明）。
口语：Agent「看见了什么」。

7. 思考（处理）

理解意图、拆解子任务、权衡方案、决定要不要调用工具、调用哪一个。
口语：Agent「琢磨怎么做」。—— 可与后续「主循环 / 规划」轨道强关联。

8. 行动（执行）

生成内容、修改资源、调用 API/命令行/插件等。
口语：Agent「真去做了」。—— 可与「工具调用 / 系统集成」轨道强关联。

9. 反馈（调整）

根据工具输出、报错、测试结果、人类反馈，更新计划或纠错。
口语：Agent「根据结果改路线」。

为什么Agent可以在一定范围内工作：

(1) 上下文理解

- 含义：能把任务描述与环境信息（文档、代码、历史步骤）放在一起理解。
- 例子：「重构登录模块」需要同时理解相关文件、依赖与接口约束，而不是只改一个函数名。

(2) 目标拆解

- 含义：把大目标拆成可验证的步骤，并知道何时算完成。
- 例子：「上线前修复某缺陷」→ 定位 → 修改 → 自测/跑测试 → 检查副作用 → 汇总变更说明。

(3) 工具调用的基础逻辑

- 含义：知道有哪些能力可用、何时调用、参数怎么填、结果如何解读与纠错。
- 例子：需要读仓库结构时走搜索/读文件类能力；需要验证时走测试/构建类能力。

二、Claude Code Agent

Claude Code $\neq$ Prompt + Tools，而是 Claude Code = Agent System。它根本不是一个“大模型+几段提示词+几个脚本”的简单拼接，而是一个高度工程化的 **AI 操作系统**。其核心依托动态 Prompt 编排、职责强制拆分、破坏性验证、工具治理及全局生命周期管理五大核心设计，实现了从简单 Demo 到工业级产品的跨越，构建起高效、可靠的智能体系统。

Anthropic泄露ClaudeCode源码

顶级Agent的底层设计

Claude Code = Prompt + Tools?

Claude Code = Agent System



Claude Code: 终端原生的“智能化执行”

2. 核心支柱 型越强

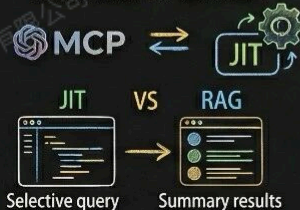
A. 终端原生 & Agent-first

Unix 哲学, 智能体主导, 并行代理 (跨文件复杂重构)

- 问题: GPT-4o 1K → 32K, 准确率 99.3% → 69.7%。

B. 即时上下文 (JIT Context)

MCP 按需加载 (Code-as-Query), Token 效用 ↑, 应对溢出



1. 智能体循环 (Agentic Loop)



Key insight: 带验证 steps 成功率 ↑

意图定义 → (OpenSpec) 果验证 → 知识沉淀 (OpenSpec)

Cursor: 编辑器优先 (IDE 插件辅导) RAG 机制, 实时补全

3. 哲学对比

Claude Code: 终端原生 (CLI 代理) JIT / Loop 机制, 自动化任务

C. 持久化记忆

CLAUDE.md (项目操作手册, 架构决策)
MEMORY.md (自动化调试心得, 越用越懂你)
Agent-first, Unix 哲学, 按需加载 (JIT), 代码即查询

D. 核心功能

Code Review (自动 Bug 追踪)
--dangerously-skip-permissions (高自主权)
自动验证 (Context)



衔接 Part 1

Part 1 已经带大家建立了 Agent 的核心认知：

- **Agent 是什么**：能自主感知、思考、行动、反馈的闭环系统
- **核心运行原理**：感知输入 → 逻辑处理 → 执行操作 → 优化调整
- **为什么能"自主工作"**：上下文理解 + 目标拆解 + 工具调用

接下来，我将用 Claude Code 的真实源码，拆解 Agent 内部到底由哪些部分组成，这些组成部分是如何协同工作的，最后通过产品对比帮大家理解 Claude Code 的定位。

一个具体场景：想象你给新来的同事安排了一个任务

在进入架构之前，先想象这个场景：

你跟一个新同事说："帮我整理一下项目里的配置文件，有些写法太旧了，改成新标准，改完跑一遍测试确认没问题。"一个靠谱的老员工会怎么做？先打开文件看看现状，想想哪些地方该改，一个个改好，跑测试验证，发现有报错再修——可能要来回好几轮，但全程不需要你在旁边盯着。Claude Code 做的完全一样——只不过它一秒钟能看完几千行代码，改完自动跑测试，发现问题立刻修。聊了太久记不住之前说了什么？它会自动"做笔记"把关键信息留下来。一个人忙不过来？它会"叫同事帮忙"分头干。

用程序员的话来说，你在终端输入 `帮我重构 utils/parser.ts 中的 parseConfig 函数`，Claude Code 会自主读取文件、分析代码、写出新版本、运行测试、自动修复 lint 错误——全程无需干预。中间上下文超限了？多级压缩自动介入。任务太复杂？模型自主 fork 出子 Agent 并行处理。

这个看似简单的交互，涉及了 Claude Code 的**每一层架构**。我们接下来就按这个故事，由内而外地拆解。

Part 2: Agent 核心组成部分解读（25 分钟）

关键 takeaway：Claude Code Agent 由五层架构组成——内核、壳层、连接、治理、规模。每层解决一个核心问题，层层扩展，对应后续 7 条分享轨道。

一、全局架构Overview：五层模型

在看架构图之前，先建立一个直觉。把 Claude Code 想象成一家自动化餐厅：

餐厅类比	Agent 层级	解决什么问题
------	----------	--------

厨师的大脑和双手——决定做什么菜、怎么做、做坏了怎么补救	内核层	Agent 怎么"想"和怎么"做"
收银台和菜单——顾客看到什么、怎么点菜	壳层	用户怎么"用"
外卖平台和供应商——和外部世界打交道的通道	连接层	与外部怎么"连"
食品安全局 + 连锁管理总部——制定规则、管理加盟店	治理层	生态怎么"管"
连锁分店 + 培训手册——多家店同时营业、把经验写进手册	规模层	怎么"大规模跑"

有了这个直觉，再看技术架构就有了"心理挂钩"——每听到一个技术概念，都可以映射到餐厅的哪个部分。



这五层对应我们集团分享的 7 条轨道：

层级	对应轨道	一句话定位
内核	轨道 1（对话引擎）+ 轨道 3（工具系统与权限）	Agent 怎么"想"和怎么"做"
壳层	轨道 6（终端产品与命令）	用户怎么"用"
连接	轨道 7（集成与通信协议）	与外部怎么"连"
治理	轨道 4（扩展与企业管控）	生态怎么"管"
规模	轨道 2（上下文与记忆）+ 轨道 5（多智能体与运行）	怎么"大规模跑"

接下来我们由内而外逐层拆解。

二、内核层：Agent 的大脑与双手

🔍 餐厅类比：这就是厨师本人——怎么决定做什么菜、怎么用刀用火、菜做坏了怎么补救。

对应轨道 1（对话引擎）+ 轨道 3（工具系统与权限）核心文件：`src/query.ts` · `src/QueryEngine.ts` · `src/Tool.ts` · `src/tools.ts`

内核层是整个 Agent 的心脏，决定了 Agent "怎么想"和"怎么做"。

2.1 核心引擎：它是怎么"持续工作"的？

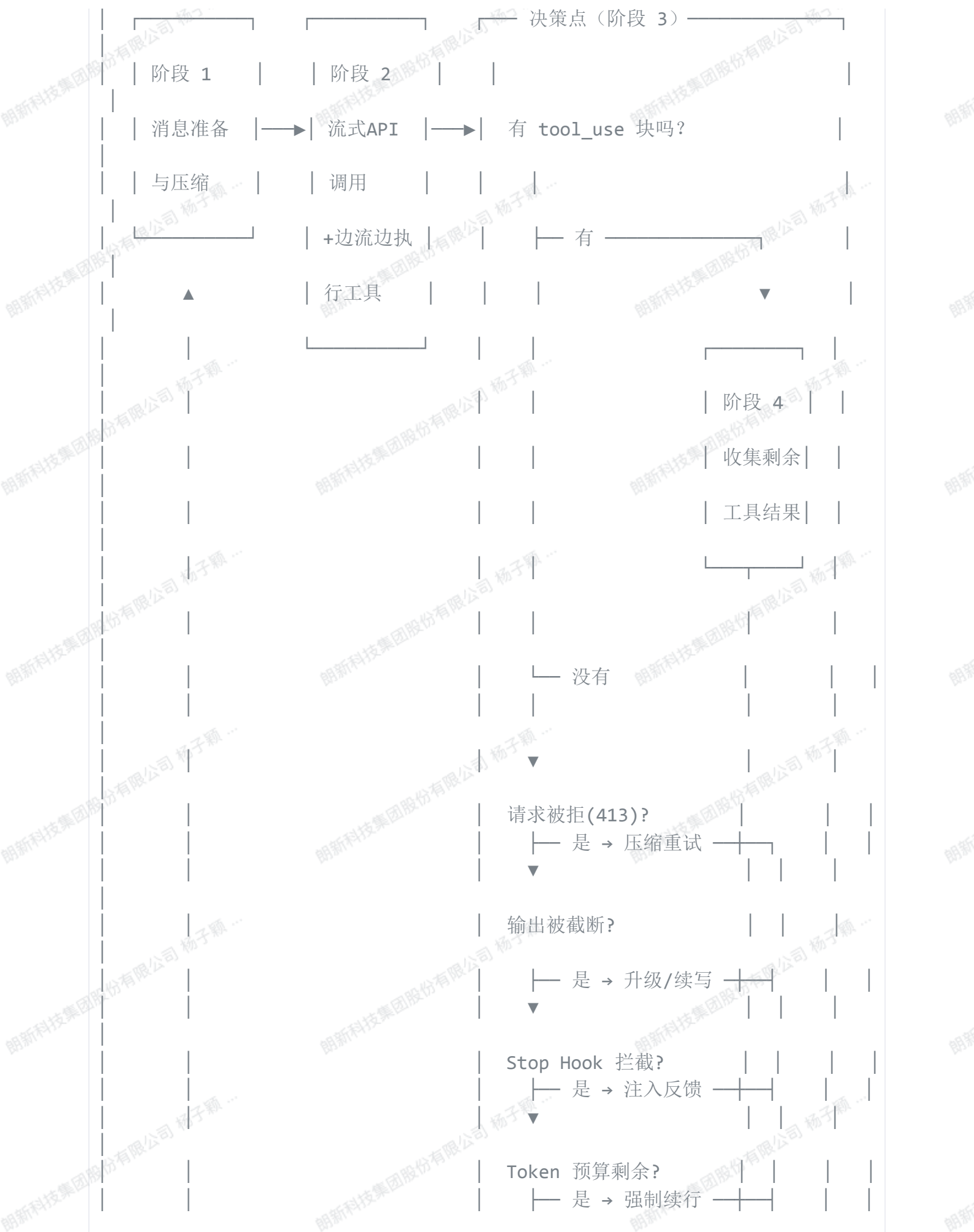
先用大白话说：Claude Code 的工作方式，就像一个永不下班的靠谱员工——做完一件事，看看下一步该干什么；如果出了问题就自己想办法补救，实在不行才找你。它不需要你告诉它每一步做什么——它自己看情况决定。

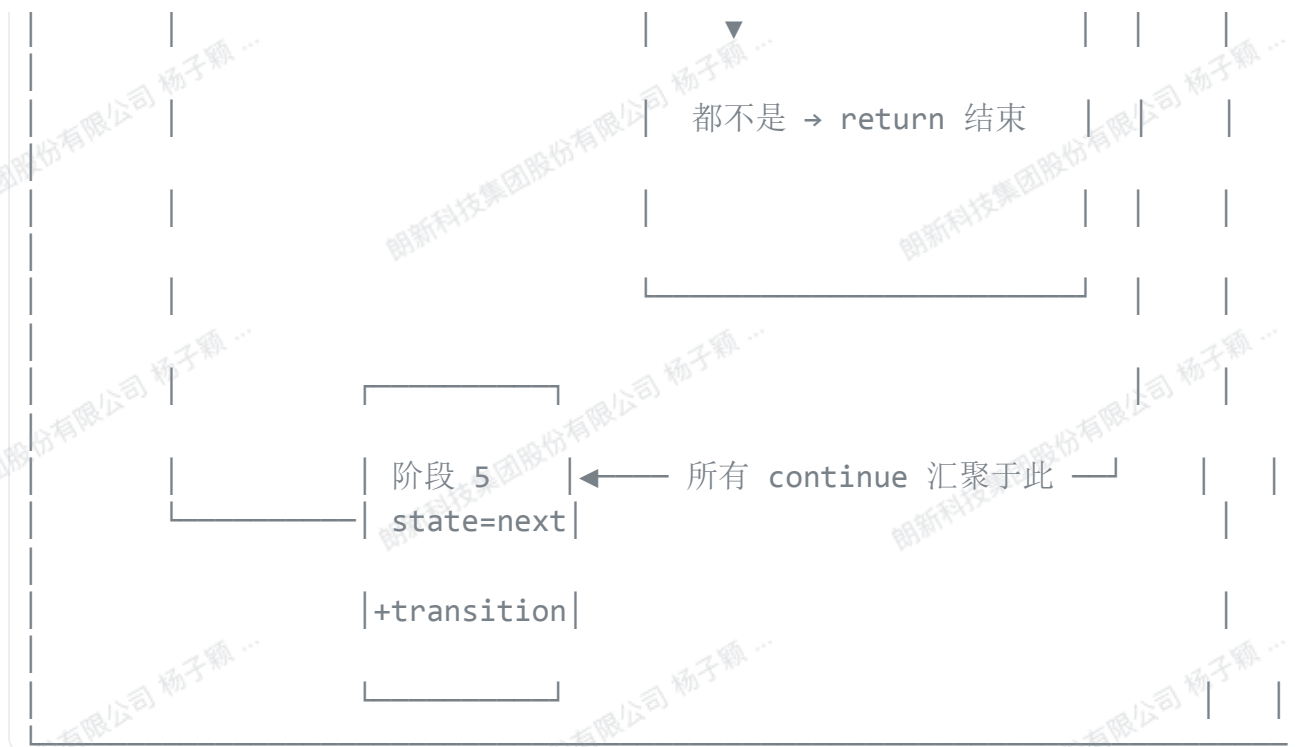
对程序员来说，这背后是一个异步生成器（**AsyncGenerator**）驱动的状态机，而非经典的 ReAct 模式（Think → Act → Observe）：

▼

复制代码

```
query() 核心循环
```





决策点的核心逻辑：和 ReAct 一样，有工具调用就进入下一轮。但 ReAct 在"没有工具调用"时直接结束，Claude Code 则会**诊断原因**——模型可能不是"做完了"，而是"被迫停下"（请求被拒、输出被截断）或"系统不满意"（Hook 拦截、预算剩余）。只有当所有恢复路径都不适用时，才真正 `return` 结束。

三个关键设计亮点：

① 死循环 + 状态赋值驱动

用 `while(true)` + `state = next` 驱动循环，而不是递归。好处是内存稳定、状态可追溯。就像一个员工的工位上始终放着一份"当前任务状态单"——做完一步就更新它，而不是每做一步就重新写一张新纸条。

② 流式执行——边说边做（`StreamingToolExecutor`）

普通 Agent 要等模型"想完"才开始"做"。Claude Code 不一样——模型**边输出边执行工具**。就像厨师不等菜单念完就开始备菜——"番茄炒蛋"四个字刚念到"番茄"就已经开始洗番茄了。

复制代码

普通 Agent（先说完再做）：

时间 →

模型输出：[===== 生成完整响应（5秒）=====]

↓ 全部生成完

工具执行：[读文件①][读文件②][编辑③]

Claude Code（边说边做）：

时间 →

模型输出: [=== 流式生成中... tool① 到达 ... tool② 到达 ... tool③ 到达 ===]

工具执行:

[读文件①]

[读文件②]

← 并行!

[编辑③] ← 串行等待

实际的 API 响应长什么样? 下面是一个真实的 Anthropic Messages API 响应结构。关键在于 `content` 数组中可以同时包含 `text` 和多个 `tool_use` 块——这就是流式执行的数据基础:

复制代码

```
{
  "id": "msg_01T8mMockAnthropicToolsA1",
  "type": "message",
  "role": "assistant",
  "model": "claude-sonnet-4-5",
  "content": [
    {
      "type": "text",
      "text": "我将调用多个工具来收集信息、检索数据并执行操作。"
    },
    {
      "type": "tool_use",
      "id": "toolu_01_weather_9xA1",
      "name": "get_weather",
      "input": {
        "location": "San Francisco, CA",
        "unit": "celsius",
        "include_forecast": true,
        "days": 3
      }
    },
    {
      "type": "tool_use",
      "id": "toolu_01_stock_9xA2",
      "name": "get_stock_price",
      "input": {
        "ticker": "NVDA",
        "currency": "USD",
        "include_prepost": true,
        "timeframe": "1d"
      }
    }
  ]
}
```

```

"type": "tool_use",
"id": "toolu_01_fx_9xA3",
"name": "get_fx_rate",
"input": {
  "base": "USD",
  "quote": "CNY",
  "amount": 1000
},
{
  "type": "tool_use",
  "id": "toolu_01_news_9xA4",
  "name": "search_news",
  "input": {
    "query": "Anthropic tool use latest updates",
    "language": "en",
    "region": "US",
    "limit": 10,
    "sort_by": "relevance"
  },
{
  "type": "tool_use",
  "id": "toolu_01_web_9xA5",
  "name": "web_search",
  "input": {
    "query": "Anthropic Messages API tool_use example",
    "top_k": 5
  },
{
  "type": "tool_use",
  "id": "toolu_01_db_9xA6",
  "name": "query_orders",
  "input": {
    "customer_id": "cust_102938",
    "status": ["paid", "shipped", "delivered"],
    "created_after": "2026-01-01",
    "limit": 50,
    "sort": { "field": "created_at", "direction": "desc" }
  },
{
  "type": "tool_use",
  "id": "toolu_01_crm_9xA7",

```

```

"name": "get_customer_profile",
"input": {
  "customer_id": "cust_102938",
  "include_tags": true,
  "include_ltv": true,
  "include_support_summary": true
}
},
{
  "type": "tool_use",
  "id": "toolu_01_email_9xA8",
  "name": "search_emails",
  "input": {
    "from": "billing@example.com",
    "subject_contains": "invoice",
    "has_attachments": true,
    "limit": 20
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_calendar_9xA9",
  "name": "find_calendar_events",
  "input": {
    "start_time": "2026-04-07T00:00:00Z",
    "end_time": "2026-04-14T00:00:00Z",
    "participants": ["alice@example.com", "bob@example.com"],
    "include_private": false
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_maps_9xB0",
  "name": "get_route",
  "input": {
    "origin": "SFO Airport",
    "destination": "1 Market St, San Francisco, CA",
    "mode": "driving",
    "alternatives": true,
    "avoid_tolls": false
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_inventory_9xB1",

```

```
"name": "check_inventory",
"input": {
  "sku_list": ["SKU-IPH15-128-BLK", "SKU-IPH15-256-BLU", "SKU-
IPADAIR-128-GRY"],
  "warehouse_ids": ["wh_us_west_1", "wh_us_central_2"],
  "include_reserved": true
},
{
  "type": "tool_use",
  "id": "toolu_01_ship_9xB2",
  "name": "track_package",
  "input": {
    "carrier": "UPS",
    "tracking_number": "1Z999AA10123456784",
    "include_history": true
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_support_9xB3",
  "name": "search_support_tickets",
  "input": {
    "customer_id": "cust_102938",
    "priority": ["high", "urgent"],
    "status": ["open", "pending"],
    "limit": 25
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_sql_9xB4",
  "name": "run_sql",
  "input": {
    "database": "analytics",
    "query": "SELECT date, revenue, orders FROM daily_metrics WHERE
date >= '2026-04-01' ORDER BY date ASC LIMIT 30;"
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_fs_9xB5",
  "name": "read_file",
  "input": {
    "path": "/data/reports/q2_plan.md",
```

```
"encoding": "utf-8",
"max_bytes": 20000
},
{
  "type": "tool_use",
  "id": "toolu_01_write_9xB6",
  "name": "write_file",
  "input": {
    "path": "/tmp/output/summary.json",
    "content": "{\n  \"status\": \"draft\",\n  \"generated_by\":\n    \"claude\"\n}",
    "overwrite": true
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_http_9xB7",
  "name": "http_request",
  "input": {
    "method": "POST",
    "url": "https://api.example.com/v1/notify",
    "headers": {
      "Content-Type": "application/json",
      "X-Request-Id": "req_abc123"
    },
    "json": {
      "event": "report_ready",
      "report_id": "rpt_20260407_001",
      "priority": "normal"
    },
    "timeout_ms": 10000
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_translate_9xB8",
  "name": "translate_text",
  "input": {
    "text": "Hello, this is a tool use example.",
    "source_lang": "en",
    "target_lang": "zh-CN",
    "preserve_formatting": true
  }
},
}
```

```

{
  "type": "tool_use",
  "id": "toolu_01_embed_9xB9",
  "name": "embed_text",
  "input": {
    "texts": [
      "Anthropic tool use example",
      "Messages API tool_use block",
      "tool_result follow-up message"
    ],
    "model": "text-embedding-v1",
    "truncate": "END"
  }
},
{
  "type": "tool_use",
  "id": "toolu_01_rerank_9xC0",
  "name": "rerank_documents",
  "input": {
    "query": "Anthropic API tool call response format",
    "documents": [
      { "id": "doc1", "text": "How tool use works in Anthropic Messages API." },
      { "id": "doc2", "text": "General overview of language model APIs." },
      { "id": "doc3", "text": "Structured examples of tool_use and tool_result." }
    ],
    "top_n": 2
  }
},
{
  "stop_reason": "tool_use",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 1832,

```

流式执行的代码实现：在 `query.ts` 的流式循环中，每收到一条流式消息，如果包含 `tool_use` 块，立刻交给 `StreamingToolExecutor.addTool()` 开始执行：

▼ 复制代码

```

// query.ts 流式接收循环（简化）
for await (const message of deps.callModel({...})) {
  yield message // 流式输出给 UI

```

```

if (message.type === 'assistant') {
  const toolBlocks = message.message.content.filter(c => c.type ===
'tool_use')
  for (const toolBlock of toolBlocks) {
    streamingToolExecutor.addTool(toolBlock, message) // ← 立刻开始执
行!
  }
}
}

```

`addTool()` 内部立刻调用 `processQueue()` 尝试执行, 同时根据每个工具的 `isConcurrencySafe` 属性做智能并发控制:

- 多个读文件/搜索 (`isConcurrencySafe = true`) → 并行执行
- 写文件/执行命令 (`isConcurrencySafe = false`) → 独占串行
- 某个 Bash 出错 → 通过 `siblingAbortController` 立刻取消所有并行兄弟任务

实际效果: 如果模型一次要读 5 个文件再做 1 次编辑, 普通 Agent 要等模型输出完才开始读文件, 而 Claude Code 在模型还在输出第 3 个 read 指令时, 前 2 个 read 已经在并行执行了。

③ 出问题不慌——多种自动恢复策略

不同于 ReAct 的"有工具调用就继续、没有就结束", Claude Code 在模型没有返回工具调用时, 会诊断原因再决定是否结束:

出了什么问题	它怎么自救	生活类比
上下文太长装不下了 (API 返回 413)	Context Collapse 排水 / Reactive Compact	笔记本写满了, 整理出精华重新抄
输出写到一半被截断了 (max_tokens)	升级限制 (8K→64K) / 注入"请继续"提示	话说到一半被打断, "你接着说"
Stop Hook 拦截 (外部检查发现问题)	注入错误信息让模型重新处理	质检员打回重做
Token 预算还剩余	强制续行	今天工时还没用完, 继续干活
模型 API 调用失败	自动降级到备用模型重试	主厨临时缺席, 副厨顶上
以上都不是	才真正结束	确认做完了, 下班

本质上前三种是 error recovery（错误恢复），后两种是 policy enforcement（策略执行）——模型没出错，但系统主动"不满意"，强制续命。

🔍 技术深潜：State 类型与恢复路径全景（点击展开）

直接来自 `src/query.ts:204-217`：

⌵ 复制代码

```
type State = {  
  
  messages: Message[ ]  
  
  toolUseContext: ToolUseContext  
  autoCompactTracking: AutoCompactTrackingState | undefined  
  maxOutputTokensRecoveryCount: number // 输出截断恢复计数  
  hasAttemptedReactiveCompact: boolean // 是否已尝试响应式压缩  
  maxOutputTokensOverride: number | undefined // 输出 token 上限覆盖  
  pendingToolUseSummary: Promise<ToolUseSummaryMessage | null> |  
  undefined  
  stopHookActive: boolean | undefined // Stop 钩子是否激活  
  turnCount: number  
  transition: Continue | undefined // 上一次循环继续的原因（可追溯）  
}
```

`transition` 字段是调试关键：每次循环继续时都会记录原因（`Continue` 类型含 `reason` 字段），使得恢复路径完全可追溯。

恢复/续行路径全景（来自 `query.ts` 中所有 `transition` 和 `return` 语句）：

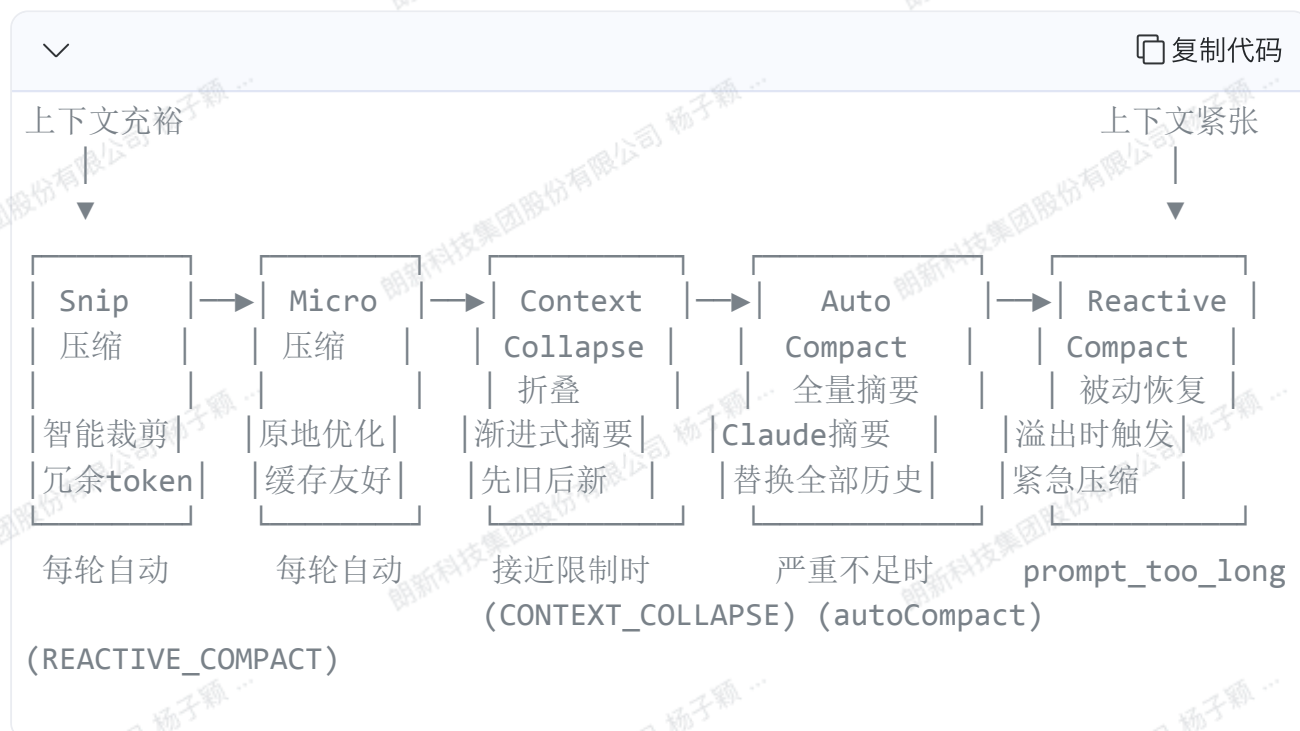
续行（ <code>state = next; continue</code> ）	终止（ <code>return</code> ）
<code>collapse_drain_retry</code> — Context Collapse 排水重试	<code>completed</code> — 正常完成
<code>reactive_compact_retry</code> — 响应式压缩重试	<code>aborted_streaming</code> — 用户中断流式
<code>max_output_tokens_escalate</code> — 输出限制升级（8K→64K）	<code>aborted_tools</code> — 用户中断工具执行
<code>max_output_tokens_recovery</code> — 注入"继续"提示重试	<code>prompt_too_long</code> — 压缩后仍超限

<code>stop_hook_blocking</code> — Stop 钩子阻塞	<code>model_error</code> — API/运行时错误
<code>token_budget_continuation</code> — Token 预算尚余	<code>image_error</code> — 图片大小错误
<code>next_turn</code> — 正常下一轮（有工具调用）	<code>stop_hook_prevented</code> — 钩子阻止继续
	<code>hook_stopped</code> — Post-Tool 钩子终止
	<code>blocking_limit</code> — Token 阻塞限制
	<code>max_turns</code> — 达到最大轮次

2.2 上下文压缩：它怎么做到"聊再久也不忘事"？

先用大白话说：你跟同事聊一个项目，聊了三个小时，你不可能记住每一句话。你会自动做笔记——把重要的结论和决策记下来，忘掉那些"嗯嗯好的OK"之类的废话。Claude Code 也一样——它的"记忆"有容量上限，所以它有一套自动"做笔记"的系统。

对程序员来说，这是一套多级渐进式压缩系统——5 层机制串联成管道，每层输出是下一层的输入，越靠后代价越大：



用笔记本的类比来理解这五级：

级别	做了什么	调 LLM?	笔记本类比
----	------	--------	-------


```
if (compactionResult) messagesForQuery =
  buildPostCompactMessages(compactionResult)

// 第 5 层 - 不在此处，在决策点 413 恢复分支中：
// reactiveCompact.tryReactiveCompact({...})
```

三个核心设计意图：

1. **层层递进，代价递增**：前两层不调 LLM（几乎零成本），后三层才可能调 LLM（有 API 开销）。绝大多数对话只触发前两层，长对话才逐步进入后面的层级。
2. **缓存友好**：Microcompact 的整个设计围绕 Anthropic 的 prompt cache——缓存编辑路径通过 `cache_edits` API 在不改变本地消息的前提下删除旧内容，确保缓存前缀字节不变、命中率最大化。
3. **Context Collapse 优先于 Auto Compact**：源码注释中明确说明的刻意设计。Collapse 是渐进式的（先折叠最旧的轮次），保留近期的细粒度上下文；Auto Compact 是全量替换（一条摘要吃掉所有历史）。如果 Collapse 就够了，就不触发 Auto Compact，让模型保留更多可用的原始信息。

实践价值：这就是为什么 Claude Code 能在一次对话中修改几十个文件而不退化——它不会简单截断历史，而是通过 5 层递进机制智能压缩，在成本、缓存效率和信息保留之间取得平衡。

2.3 工具系统：Agent 能做什么取决于它有什么"工具"

先用大白话说：一个人再聪明，如果没有工具就什么也做不了——你让一个厨师空手做菜试试？Agent 也一样：它的能力上限 = 它有多少工具。Claude Code 有 48 种以上的"工具"：读文件、写代码、跑命令、搜网页、叫同事帮忙。

对程序员来说，每个工具都是一个完整的生命周期管理单元，走完 7 步管道才能执行：



复制代码

工具执行管道（7步）

1. 工具查找 → 2. 输入解析 (Zod) → 3. 自定义验证
|
4. Pre-Tool 钩子 → 5. 权限检查 → 6. 实际执行 → 7. Post-Tool 钩子

工具全家福：

类别	代表工具	能力	生活类比
读取/搜索	FileRead, Glob, Grep, LSP	感知世界	👁️ 看文件、搜代码 —— "眼睛"

写入	FileEdit, FileWrite, NotebookEdit	改变世界	✍️ 改文件、写代码 ——"双手"
执行	Bash, PowerShell	运行命令	⚡ 跑脚本、装依赖 ——"去跑个腿"
网络	WebFetch, WebSearch	访问互联网	🌐 查文档、搜资料 ——"上网查一下"
子Agent	AgentTool, SendMessage	分身协作	👥 叫同事帮忙 ——"你去处理那个"
元操作	EnterPlanMode, AskUserQuestion	调节自身	🤔 先想想再动手 / 拿不准先问你

并发策略：哪些工具可以同时用？

并发决策的标准是每个工具的 `isConcurrencySafe` 属性。关键点："只读"不等于"可以并行"——比如一条 Bash 命令 `cd /some/path` 虽然不改文件，但会影响后续命令的工作目录，所以不能并行。

- `isConcurrencySafe === true` → 并行执行（最多 10 个一起跑）
- `isConcurrencySafe === false` → 排队串行（一个做完再做下一个）

框架会将连续的 `isConcurrencySafe` 工具分为一个批次并行，遇到非安全工具则切换为串行。

权限系统：为什么不能让 Agent 想干什么就干什么？

和你给实习生设权限一样——有些事可以放手让他做，有些事要先问你确认，有些事绝对不能碰。

权限档位	一句话解释	谁会用
<code>default</code>	每次操作都问你	谨慎的用户
<code>acceptEdits</code>	改文件自动通过，但跑命令还是问	大多数开发者
<code>bypassPermissions</code>	基本全部放行	完全信任 Agent 的高级用户
<code>dontAsk</code>	不确定的一律拒绝，无弹窗打扰	CI/CD 自动化场景
<code>plan</code>	特定条件下放行	Plan 模式使用

🔍 技术深潜：完整权限模式（点击展开）

权限系统有 5 种外部模式 + 2 种内部模式（`src/types/permissions.ts`）：

模式	行为
<code>default</code>	完整规则匹配 + 工具级权限检查 + 交互式询问
<code>plan</code>	与 <code>bypass</code> 类似的放行，但需满足特定条件
<code>bypassPermissions</code>	工具级检查后直接放行（部分路径仍不可绕过）
<code>acceptEdits</code>	自动接受编辑类操作
<code>dontAsk</code>	<code>ask</code> → <code>deny</code> ，不弹出交互式询问
<code>auto</code> （内部）	结合 <code>transcript classifier</code> 自动判断
<code>bubble</code> （内部）	Fork 子 Agent 将权限决策冒泡给父 Agent

决策结果有四种：`allow` / `ask` / `deny` / `passthrough`（由工具返回，框架视为 `ask`）。

三、壳层：用户看到什么、怎么操作

🔍 餐厅类比：收银台和菜单——顾客不需要知道厨房怎么运作，只需要看菜单、下单、等上菜。

对应轨道 6（终端产品与命令）核心文件：`src/main.tsx` · `src/screens/REPL.tsx` · `src/commands/` · `src/components/`

壳层决定了用户的交互体验。Claude Code 不只是“终端里的聊天机器人”——它是一个能在你电脑上直接动手的 AI 同事，能读你的代码、改你的文件、跑你的命令。这带来了传统工具从未面对的交互设计挑战：如何让一个能“动手”的 AI 既好用又可控？

3.1 三种使用姿势：一个引擎，多种入口

不同场景用不同方式启动，但背后是同一个 Agent 引擎：

姿势	命令示例	适合场景
交互对话	<code>claude</code>	日常开发，边聊边做

一句话任务	<code>claude "帮我重构这个函数"</code>	明确任务，不需要来回对话
管道/脚本	<code>git diff claude -p "review this"</code>	CI/CD、批处理、自动化流水线

还可以通过 flag 精细控制：`--model opus`（切换模型）、`--max-budget-usd 2`（限制花费）、`-w feature`（隔离的 git worktree 分支开发）、`-c`（继续上次对话）。

3.2 交互范式：不只是打字聊天

Claude Code 的输入方式远超传统 CLI 的"敲命令"模式：

输入方式	例子	意义
自然语言	"帮我修复这个 bug"	零学习成本，说人话就行
@ 引用文件	<code>@src/index.ts</code> 这里有问题	精确指向上下文，不用复制粘贴
! 执行命令	<code>! git status</code>	对话中随时查看系统状态
粘贴截图	<code>Ctrl+V</code> 粘贴错误截图	多模态——截图直接给 AI 看
Pipe 管道	<code>cat error.log claude -p "explain"</code>	和 Unix 哲学无缝衔接
Slash 命令	<code>/plan</code> 、 <code>/compact</code> 、 <code>/review</code>	精确触发特定功能

常用 Slash 命令速览：

命令	做什么
<code>/plan</code>	让 Agent 先出方案再动手（复杂任务必用）
<code>/compact</code>	手动压缩对话历史（对话太长时用）
<code>/model</code>	切换模型（Sonnet 快而便宜，Opus 最聪明）
<code>/memory</code>	查看和管理 Agent 的项目记忆（CLAUDE.md）

/mcp	管理外部工具连接
/review	代码审查
/cost	查看当前会话花了多少钱

3.3 三种信任模式：Shift+Tab 一键切换

这是 Claude Code 最核心的产品设计——用一个快捷键在三种人机协作模式间切换：

复制代码

Shift+Tab 循环切换

Normal（默认）

→

Auto-Accept

→

Plan

每步都问你

全部放行

只看不做

模式	行为	适用场景
Normal	Agent 每修改一个文件、每跑一条命令都会问你"可以吗？"	默认模式，完全掌控
Auto-Accept	Agent 自主执行，不再询问	信任度高的重复性任务
Plan	Agent 只读代码、只出方案，不动任何东西	复杂任务先看方案再决定

推荐 workflow：Plan 模式理解问题 → 审查方案 → 切换 Normal/Auto-Accept 执行。

3.4 后悔药：Rewind 机制

AI 改错了怎么办？Esc Esc 打开回退菜单：

回退选项	效果
只回退对话	代码保持不变，对话回到之前的某个点
只回退代码	对话保持不变，文件恢复到之前的状态
全部回退	对话和代码一起回到之前的某个点

这解决了用户最大的焦虑："万一 AI 改坏了呢？"——随时可以一键回退。

实践价值：Claude Code 的壳层设计证明，Agent 产品的交互核心不是"界面好不好看"，而是如何在 **AI 自主性和用户控制感**之间找到平衡。三种信任模式 + Rewind 后悔药，让用户敢把方向盘交给 AI。

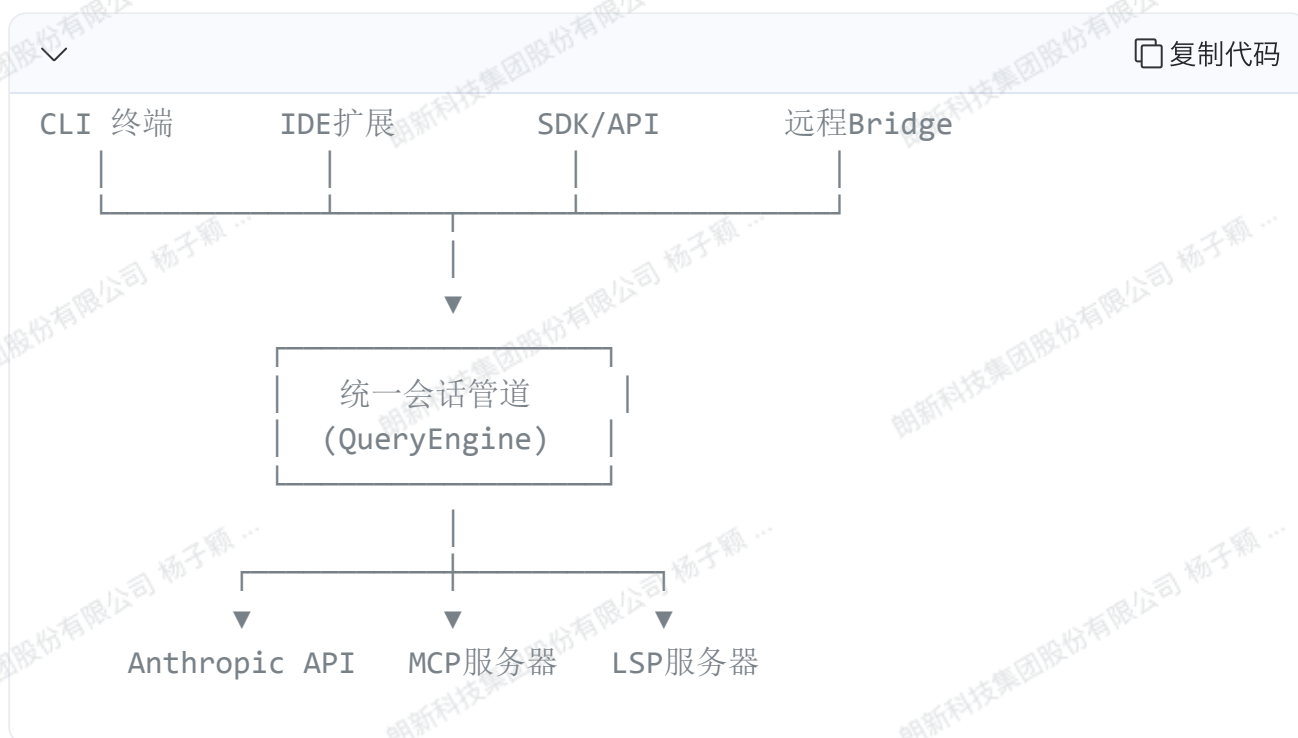
四、连接层：Agent 怎么和外部世界打交道

🔍 **餐厅类比：**外卖平台和供应商——餐厅再好，如果食材进不来、外卖送不出去，就是一座孤岛。

对应轨道 7（集成与通信协议）核心文件： `src/services/api/` · `src/services/mcp/` · `src/c/bridge/` · `src/cli/transport/`

Agent 再聪明，如果连不上外部系统就什么都做不了。连接层让 Claude Code 能对接你的数据库、Jira、Slack、Chrome——从"只会写代码"变成"能操作一切"。

4.1 不管你从哪里来，最终都进入同一条管道



无论你是在终端直接输入、还是在 VS Code 里用、还是通过 API 调用，最终都汇入同一个引擎——就像一家餐厅不管是堂食、外卖还是团购，后厨都是同一个。

4.2 MCP 协议：Agent 世界的"USB 接口"

MCP (Model Context Protocol) 是 Claude Code 与外部工具连接的标准协议。最简单的理解：**MCP 就是 AI 的 USB 接口**——定义了一个标准插口，任何工具只要实现这个接口就能"插上"Agent 使用。

- MCP 工具与内置工具共享同一个 `Tool` 类型接口，合并到统一工具池——Agent 不区分对待
- 命名规范：`mcp_{服务器名}_{工具名}`
- 支持 5 种传输方式：`stdio` / `SSE` / `HTTP` / `WebSocket` / `SDK`
- 运行时动态发现，随插随用

实践价值：通过 MCP，一个 Claude Code Agent 可以连接 Chrome DevTools、数据库、Jira、Slack 等任何外部系统，能力可以无限扩展。

五、治理层：谁来管住 Agent？

🔍 **餐厅类比：**食品安全局 + 连锁管理总部——制定统一标准、检查合不合规、管理所有分店。

对应轨道 4（扩展与企业管控）核心文件： `src/skills/` · `src/plugins/` · `src/utils/set`
`tings/` · `src/services/policyLimits/`

公司不可能让一个不受管控的 AI 随便跑。治理层解决的是**企业信任问题**——就像给新员工设定系统权限和行为规范。

它回答两个核心问题：**如何教 Agent 新技能**，以及**如何确保它在企业里安全运行**。

5.1 五层扩展体系：从零件到生态

Claude Code 的扩展能力不是一个扁平的列表，而是**有层次的**——每一层解决不同的问题：

			复制代码
你要连外部系统？	→ MCP	（插座——负责连接）	
你要在流程节点插逻辑？	→ Hook	（安检门——负责拦截和补充）	
你要给任务分专人？	→ Subagent	（专职角色——负责分工）	
你要把做事方法沉淀？	→ Skill	（操作手册——负责知识沉淀）	
你要把以上统一安装分发？	→ Plugin	（App——负责打包和分发）	

前四层是零件，**Plugin** 是收口层。它不替代其他层，而是把零散的零件打包成一个可安装、可共享、可开关的单元——就像手机 App Store 里的 App，装上就能用，不想要就关掉。

Plugin 能装什么？——完整能力清单：

组件	目录/文件	触发方式	做什么
Commands	<code>commands/*.md</code>	用户手动 / 插件名： 命令	注册 Slash 命令

Skills	<code>skills/*/SKILL.md</code>	Agent 自动判断 or 用户手动	注入领域知识和行为规范
Agents	<code>agents/*.md</code>	用户指定 or Agent 分派	定义专职角色（如安全审查员、代码评审员）
Hooks	<code>hooks/hooks.json</code>	生命周期自动触发	在关键节点拦截、检查、补充
MCP Servers	<code>.mcp.json</code>	自动启动	连接外部系统（GitHub、Jira、数据库）
LSP Servers	<code>.lsp.json</code>	自动启动	代码智能（跳转定义、类型检查、诊断）
Settings	<code>settings.json</code>	启用时自动应用	插件级默认配置

一个真实 Plugin 的解剖（kevin-plugin）：

▼ 复制代码	
<pre> kevin-plugin/ ├── .claude-plugin/ │ └── plugin.json ├── commands/ │ └── ralph-loop.md ├── hooks/ │ ├── hooks.json │ ├── pre-tool.sh │ ├── post-bash.sh │ └── stop-hook.sh ├── prompt │ └── session-end.sh ├── skills/ │ ├── remotion-best-practices/ │ ├── self-improving-agent/ │ └── skill-creator/ </pre>	← 清单：名字、版本、描述 ← Slash 命令（用户手动触发） → /kevin-plugin:ralph-loop "启动迭代循环" → /kevin-plugin:cancel-ralph "取消循环" → /kevin-plugin:help "查看帮助" ← 生命周期钩子（自动触发） ← 注册 5 个钩子覆盖完整生命周期 ← PreToolUse: 工具调用前日志记录 ← PostToolUse: 工具调用后日志记录 ← Stop: Ralph Loop 在这里拦截并重新注入 ← Stop: 会话结束清理 ← Agent Skills（Agent 自动调用的知识） ← 写 Remotion 代码时自动加载视频制作最佳实践 ← 自我改进的多层记忆系统（语义+情景+工作记忆） ← Skill 创建和评测工具

- └─ workflow-orchestration-patterns/ ← Temporal 分布式工作流知识
- └─ agents/ ← 专职角色定义 (Agent 分派或用户指定)
 - └─ security-reviewer.md ← 专职安全审查: 聚焦漏洞扫描和风险评估
 - └─ test-writer.md ← 专职测试编写: 根据代码变更自动补充测试
- └─ scripts/ ← 辅助脚本 (被 hooks 和 commands 调用)
 - └─ bootstrap-runtime.mjs ← SessionStart 时初始化记忆系统运行时
 - └─ auto-recall.mjs ← UserPromptSubmit 时自动召回相关记忆
 - └─ auto-capture.mjs ← Stop 时自动捕获本次会话经验
 - └─ ...

安装后的实际体验:

组件	行为	用户感知
Hooks	自动生效	每次会话自动初始化记忆、自动召回上下文、自动捕获经验——用户无感
Skills	按需激活	写 Remotion 代码时 Agent 自动获得视频制作知识——用户无感
Agents	按需分派	复杂任务时 Agent 自动调用 security-reviewer 做安全审查——用户无感
Commands	手动触发	<code>/kevin-plugin:ralph-loop</code> <code>"Fix auth bug" --max-iterations 10</code> ——用户主动调用

Plugin 的管理方式（类似 App Store）：

▼ [复制代码](#)

```
# 添加 Marketplace（类似添加应用商店）
claude plugin marketplace add anthropics/claude-code-plugins

# 浏览和安装（类似下载 App）
/plugin install github@claude-plugins-official

# 启用/禁用（类似开关 App）
/plugin disable github@claude-plugins-official
/plugin enable github@claude-plugins-official
```

本地开发测试（类似 TestFlight）

```
claude --plugin-dir ./my-plugin
```

Plugin 的核心价值：把"一个人电脑上调顺的一套东西"变成"团队可以一起装的一套东西"。以前你得逐项教团队配 hook、配 skill、配 MCP——现在只需要说"装这个 Plugin"。

5.2 四层配置叠加：企业怎么管



实践价值：IT 部门可以通过设备管理（MDM）强制策略，项目组可以定制规则，开发者仍保有个人偏好——和公司管理电脑软件一个道理，层层叠加，高级别覆盖低级别。

六、规模层：一个 Agent 忙不过来怎么办？

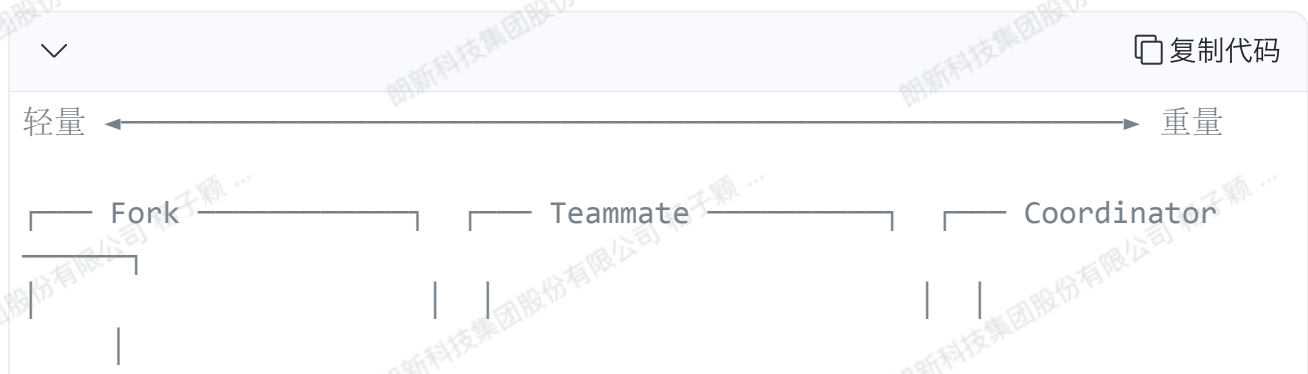
🔍 **餐厅类比：**开连锁分店 + 把经验写进培训手册——一家店忙不过来就开多家，好的做法写成 SOP 让所有店共享。

对应轨道 2（上下文与记忆）+ 轨道 5（多智能体与运行）核心文件：`src/coordinator/`

- `src/Utils/swarm/`
- `src/memdir/`
- `src/services/extractMemories/`

规模层解决两个问题：一个人忙不过来怎么办和怎么把经验留下来。

6.1 三种"叫帮手"的方式





	Fork	Teammate	Coordinator
一句话	分身去查个东西，查完就消失	叫几个同事分头干活，各管各的	项目经理统一分配任务
生活类比	你打开浏览器新标签页快速搜个答案，然后关掉	团队各自领任务，互不干扰，微信群里同步进度	项目 PM 建了看板，给每个人分 ticket
触发方式	模型调用 AgentToo1	模型调用 TeamCreat eTool	环境变量开启 Coordinator 模式

通信方式	函数返回值（同步）	Mailbox 文件系统邮箱（异步）	Scratchpad 共享文件
适用场景	简单子任务、需要父上下文	并行研究、代码审查	复杂多步骤项目、集中调度

设计哲学：模型自己决定要不要"叫帮手"、叫几个、怎么分工（通过工具调用 `AgentTool`、`TeamCreateTool`）。框架负责真正的调度——并发控制、权限传递、状态隔离。

Fork 核心机制：

- **Prompt Cache 共享：**Fork 子 Agent 复制父 Agent 的完整对话历史和 system prompt（字节级精确复制），所有 Fork 子 Agent 共享同一份 Prompt Cache，极大节省 token 成本
- **防递归机制：**通过检测对话历史中的 `<fork-boilerplate>` 标签来阻止子 Agent 再次 Fork
- **权限冒泡：**`permissionMode: 'bubble'`，子 Agent 的权限请求冒泡到父 Agent 终端
- **Worktree 隔离：**可在独立的 Git Worktree 中执行，不影响父 Agent 的文件状态

Teammate 核心机制：

- **后端自动检测：**按优先级选择执行后端：tmux 分屏 → iTerm2 → 外部 tmux → InProcess 回退
- **Mailbox 通信：**文件系统上的邮箱文件，解耦的异步通信
- **团队生命周期：**团队持续存在直到主动解散，可被反复发消息执行新任务

Coordinator 核心机制：

- **工具集缩减：**只有 `AgentTool`、`SendMessageTool`、`TaskStopTool`，没有 Bash/Read/Write/Edit——只能调度，不能动手
- **四阶段工作流：**Research（Worker 并行调查）→ Synthesis（Coordinator 自己理解并写规格）→ Implementation（Worker 执行）→ Verification（Worker 验证）
- **Scratchpad 共享目录：**Worker 之间传递大量数据的通道

协作架构中的内置角色——Agent 能派谁去干活？

三种架构是"怎么组队"的问题，而**内置角色**是"派谁去"的问题。框架预定义了 6 种专职角色，每种有严格的工具权限边界：

角色	定位	工具权限	典型场景
general-purpose	通用多面手	全部工具	需要"动手"的子任务
explore	只读搜索专家	Glob/Grep/Read/Bash(只读)，禁用 Edit/Write/Agent	探索陌生代码库、分析架构

plan	规划师	与 explore 相同（只读）	先看方案再动手
verification	验证专家	Bash(只读)+浏览器自动化，禁用 Edit/Write	找 bug 而不是确认能用——跑测试、检查边界
claude-code-guide	CC 使用指南	Read/Glob/Grep/WebFetch/WebSearch	回答关于 Claude Code 功能的问题
statusline-setup	终端状态栏	Read/Edit	设置终端状态栏配置

Verification 角色的对抗性设计值得展开：它的 system prompt 中明确列出了 Agent 容易犯的两种认知偏差：

1. **验证回避**：面对检查时找理由不运行——“读代码不是验证，运行它”
2. **被前 80% 迷惑**：看到测试通过就倾向于 PASS——“你的全部价值在找到最后 20%”

三种架构 × 六种角色，组合出从“快速查一个问题”到“多人专业团队攻坚大项目”的全部协作场景。

6.2 记忆系统：怎么把经验留下来

Agent 不能是“金鱼记忆”——今天帮你改完代码，明天就忘了你的编码风格。Claude Code 有四个阶段的记忆保存：

阶段	做了什么	生活类比
对话中（Session Memory）	多级上下文压缩保持对话可用（详见 2.2 节）	开会时在本子上随手记要点
对话结束（Extract Memories）	用 forked agent 自动提炼持久化记忆	会后整理会议纪要
跨会话（Auto Dream）	定期整理合并多次对话的记忆（24h + 5 次会话门控）	周末整理这周所有的会议纪要，合并成项目文档
团队级（Team Memory Sync）	通过 HTTP 推拉在团队成员间同步	把文档放到共享 Wiki 上，全团队都能看到

存储：文件系统即数据库



复制代码

```
~/ .claude/projects/<git-root>/memory/
```

```
|— MEMORY.md          ← 索引 (≤200行, 每行一个指针, 非内容本身)
|— user_role.md       ← 独立话题文件 (带 frontmatter 类型标注)
|— feedback_testing.md
|— project_auth_rewrite.md
|— team/              ← 团队记忆 (Team Memory Sync 管理)
```

四种封闭类型：**user**（角色偏好）、**feedback**（纠正与确认）、**project**（决策背景）、**reference**（外部系统指针）。MEMORY.md 只是索引，实际知识分散在各话题文件中。

召回：LLM 语义选择，不只是 grep

`findRelevantMemories` 用 **Sonnet** 模型对记忆文件的 frontmatter 清单做语义匹配，选出最相关的 ≤5 个文件。你问“部署时端口冲突”，即使记忆写的是“修改 docker-compose port mapping”，Sonnet 也能关联上。

防幻觉：不信任自己的记忆

源码中经 A/B 测试验证的措辞：“A memory that names a specific function or file is a claim that it existed WHEN THE MEMORY WAS WRITTEN. Before recommending it — check the file exists.” 记忆是时间点快照，行动前必须验证当前状态。

实践价值：Claude Code 能记住你的项目偏好、编码风格、团队约定，且这些记忆在团队成员间共享。

6.3 遥测：Agent 的“行车记录仪”

先用大白话说：Agent 的行为是不确定的——同一个问题它可能走完全不同的路径。没有“行车记录仪”，出了问题几乎无法复现和排查。

遥测系统记录 Agent 的每一步决策，就像行车记录仪：

- **链路追踪 (Trace)：**完整还原“想了什么 → 调了什么工具 → 生成了什么子 Agent”的时序——定位延迟瓶颈
- **指标统计 (Metric)：**Token 花了多少、工具调了几次、压缩触发了几回——驱动成本优化
- **事件分析 (Analytics)：**用户行为模式、错误分布——决定哪些功能该全量开放

Claude Code 的自动恢复策略（如模型降级、响应式压缩）本身就依赖这些运行数据来触发。遥测既是后视镜，也是自动驾驶的传感器。

七、Part 2 小结：五层联动

回到开头的场景——你说“帮我整理一下配置文件”。看看五层是怎么协作的：

你在终端输入任务

【壳层】"收银台"接单 → 是快捷命令还是普通对话？

【连接层】"外卖平台"建立连接 → 与 Anthropic API 流式通信

【内核】"厨师"开始工作 → query() 循环启动

—— 备料：组装上下文（系统提示词 + 项目规范 + 记忆 + Git 状态）

—— 看菜单：流式调用模型 → 模型决定先读取文件

—— 同时备多道菜：并行读取多个相关文件（isConcurrencySafe）

—— 精细操作：逐个编辑文件（串行执行 → 经过权限检查）

—— 菜做坏了？→ 多级"补救"自动介入【恢复策略】

—— 忙不过来？→ "叫帮手"分身处理【规模层】

—— 要跑测试？→ 加载 verify 技能【治理层】

—— 更新状态 → 继续循环直到完成

【壳层】把结果端上桌 → 渲染到终端

【规模层】写培训手册 → 提取记忆 · 更新 Session Memory

核心设计哲学：模型驱动工具选择，框架控制执行编排。

模型决定做什么（读哪个文件、是否需要叫帮手），框架决定怎么做（压缩策略由阈值控制、并发由 isConcurrencySafe 控制、权限由规则引擎控制、恢复由 query.ts 中的状态机逻辑控制）。不需要人工编排流程图，一个 while(true) 循环 + 统一的工具接口就足够了。

Part 3：主流 Agent 产品对比（7 分钟）

核心对比维度：原理差异、核心优势、适配场景、实践落地效果

一、三个产品，一句话搞懂

产品	一句话理解	谁最适合用
Claude Code	Anthropic 官方的"深度编码专家"——企业级、团队协作、工具最全	需要深度编码 + 团队协作 + 企业管控的团队

OpenClaw	社区打造的"全能私人助手"——跑在你自己设备上，能连 20+ 消息平台	想要一个私有化部署、覆盖生活和工作全场景的个人 AI 助手
OpenCode	终端极客的"开源编码利器"——模型自由、TUI 极致、客户端/服务器分离	想要 100% 开源、模型不受绑定、追求终端体验的开发者

二、基础信息

维度	Claude Code	OpenClaw	OpenCode
开发方	Anthropic	Peter Steinberger & 社区	anomalyco (terminal.shop 团队)
GitHub	84.6K 🌟 / 7.2K forks	350K 🌟 / 70.4K forks	139K 🌟 / 15.3K forks
开源协议	源码可读（非完全开源）	MIT	MIT
语言	TypeScript / Bun	TypeScript / Node.js	TypeScript / Bun
定位	深度编码 Agent	全能个人 AI 助手	开源编码 Agent
模型支持	Claude 系列深度优化	多模型（推荐最强最新模型）	75+ 供应商 (Provider-agnostic)

数据来源：各项目 GitHub 主页（截至 2026 年 4 月初）。

三、原理差异：三种截然不同的 Agent 范式

先用大白话说：

对程序员来说：

维度	Claude Code	OpenClaw	OpenCode
----	-------------	----------	----------

核心架构	单体 CLI, AsyncGenerator 状态机驱动	Gateway (WS 控制平面) + Pi Agent Runtime (RPC)	Client/Server 分离, TUI 前端 + TypeScript Server
Agent 循环	<code>while(true)</code> + <code>state = next</code> , 流式工具执行	Pi Agent Runtime, RPC 模式, 工具流式传输 + 块流式传输	AI SDK 标准 ReAct 循环
上下文管理	多级渐进式压缩 (Snip→Micro→Collapse→Compact→Reactive)	Session 模型 + 会话修剪 + Compaction 命令	标准 Compaction
多 Agent	Fork / Teammate / Coordinator 三档	Multi-agent routing (按渠道/账户路由到隔离 Agent)	task 工具生成子 agent
UI 技术	React + Ink (终端内组件化)	20+ 消息渠道 + WebChat + macOS/iOS/Android 原生 App	Bubble Tea TUI + 桌面 App + VS Code 扩展

一个关键区别:

	Claude Code	OpenClaw	OpenCode
核心循环在哪里跑	你的终端进程内	Gateway 控制平面 (可跑在本地或远程 Linux)	独立 Server 进程 (可远程)
你通过什么和它交互	终端 / IDE 扩展 / SDK	WhatsApp / Telegram / Slack / Web / macOS App / iOS / Android	终端 TUI / 桌面 App / VS Code / 手机 (远程)

四、核心优势对比

能力维度	Claude Code	OpenClaw	OpenCode
工具体系	48+ 内置工具 + MCP 动态扩展	浏览器控制 + Canvas + Cron +	精选核心工具 + MCP + LSP 原生

		20+ 渠道动作 + Skills 平台	
记忆系统	MEMORY.md + Session + Extract + AutoDream + Team Sync	Workspace 级 AGENTS.md/SOUL.md + Session 持久化	基础 Session 记忆
权限安全	7 种权限模式 + 规则引擎 + ML 分类器	DM Pairing (配对码验证) + Docker 沙箱 + per-session 隔离	基础 Permission 规则
扩展机制	Skills + Plugins + Hooks + MCP	Skills (bundled/managed/workspace) + Plugins + MCP	MCP + 自定义 Agent 配置
企业管控	MDM + 远程策略 + 组织限制	自托管 + Docker 沙箱 + Tailscale 远程访问	基础配置
跨平台	终端 + IDE 扩展 + Bridge	20+ 消息平台 + macOS/iOS/Android + 语音 + Canvas	终端 + 桌面 App + VS Code + Zed + Windsurf

五、适配场景分析

你的需求	推荐	核心理由
企业级深度编码，需要管控、合规、团队协作	Claude Code	多级压缩 + 三级缓存 + MDM 管控 + 成熟多 Agent + Team Memory
私人全能助手，生活 + 工作全覆盖，自部署	OpenClaw	20+ 消息平台 + 语音 + Canvas + macOS/iOS/Android + 本地运行
100% 开源 + 模型自由 + 终端极致体验	OpenCode	MIT 协议 + 75+ 供应商 + Client/Server 分离 + LSP 原生

团队协作编码	Claude Code	Teammate + Team Memory Sync + 企业策略叠加
移动端 / 多设备控制 AI	OpenClaw	iOS/Android 节点 + 远程 Gateway + Tailscale 穿透
追求轻量、启动快、高度可定制	OpenCode	精选工具 + Bun 运行时 + 灵活的 Agent 切换 (build/plan)

六、实践落地效果

维度	Claude Code	OpenClaw	OpenCode
社区活跃度	84.6K ★, Anthropic 官方持续投入	350K ★ / 70.4K forks / 1,556 贡献者, 社区极度活跃	139K ★ / 15.3K forks / 755 个发布版本, 迭代极快
更新频率	随 Anthropic 产品节奏, 稳定迭代	日级发布 (28,390 commits), 社区驱动	日级发布 (10,910 commits), 快速响应
上手门槛	中等 (需要 Anthropic API Key + 终端经验)	较高 (需要配置 Gateway + 消息渠道 + 可选的 Docker 沙箱)	低 (一条命令安装, 支持多包管理器 + Homebrew + Nix)
编码场景效率	极高——流式执行 + 多级压缩 + 自动恢复, 长任务几乎不中断	中等——编码是能力之一但非唯一焦点, 更偏全能助手	高——专注编码 + LSP 原生 + Client/Server 可远程协作
非编码场景	有限 (主要面向开发者)	极广——消息回复、日程管理、语音对话、Canvas 可视化、跨设备控制	有限 (主要面向开发者)
典型使用方式	在终端中长时间对话式编码, 修改几十个文件不退化	在手机上发条微信让 AI 查资料、在电脑上用 Canvas 画图、用语音和 AI 对话	在终端中快速编码, 用 plan agent 分析陌生代码库, 远程驱动开发

七、Claude Code 的核心竞争力

在编码深度这个维度上，Claude Code 仍然是三者中最强的：

优势	技术实现	大白话
边想边做	流式工具执行 (StreamingToolExecutor)	不等"想完"就开始干活，速度更快
聊再久也不忘事	多级上下文压缩（5 种机制 渐进介入）	自动做笔记，不会因为聊太久而丢失关键信息
省钱省时	三级 Prompt Cache（全局/ 会话/轮次）	重复的部分只算一次钱，响应更快
出问题自己修	完整状态转换图（7+ 续行 + 10+ 终止路径）	报错了自动补救，几乎不需要你手动重试
能连一切	Git 感知 + LSP + Worktree + MCP 生态	不只会写代码，还能连数据库、浏览器、Jira
企业放心用	MDM + 远程策略 + 7 种权限模式	IT 部门能管控，不用担心 AI 乱来

而 **OpenClaw** 的核心竞争力在于"无处不在"——20+ 消息渠道 + 语音 + Canvas + 移动端，让 AI 真正融入你的日常生活。**OpenCode** 的核心竞争力在于"完全开放"——100% MIT 开源 + 75+ 模型供应商 + Client/Server 架构允许远程控制，是最灵活的编码 Agent。

Part 4：Agent实践&应用

一、Agent能用来干嘛

1. 研发效能

多文件修改、测试驱动修复、重复性改动（脚手架、迁移、规范化）、文档与代码一致性维护等。

要点：强调可验证——能跑测试、能 diff、能回滚。

2. 知识与流程

检索、总结、生成草案；在受控工具下对接工单、知识库、内部 API（视治理要求而定）。

3. 自动化与编排

把「多步、易错、需上下文」的操作串成可重复流程；人在关键节点审批或确认。

二、怎么用好Agent

原则	说明
目标写清楚	产出物、约束、完成标准（例如「通过 CI」「不改变对外 API」）。
缩小权限面	最小权限、沙箱、分支/评审；避免给过大范围的生产权限。
可观测	日志、步骤痕迹、变更 diff；出问题能复盘。
人在环路	关键步骤确认；与 Part1「反馈」呼应，人是反馈源之一。
持续沉淀规范	项目说明、编码规范、目录约定——减少模型「猜」。

Agent 的价值 = 模型能力 × 工具与环境 × 流程治理；后两者往往决定能不能上生产。

三、常见应用案例

1. 晨间智能简报

配置一个 Heartbeat 技能，每天早上 7 点自动推送天气、日历、邮件摘要、行业头条。社区反馈：绝大多数用户在一周内就把这个功能变成了日常习惯。

2. 邮件自动分流

AI 自动完成分类、退订垃圾邮件、为重要邮件起草回复。

3. 手机端 DevOps

利用ADP协议，使用手机端Channel，直接审查 PR、运行测试、检查 CI/CD 状态、合并代码

4. 文件自动归档

监控下载文件夹，自动将 PDF、图片、安装包按类型和主题分类归档。一条命令即可完成。

5. 个人 CRM 系统

从 Gmail、日历、会议记录中自动提取联系人信息，维护人际关系网络，提醒跟进事项。

6. 多 Agent 营销自动化

多个 Agent 协作：选题调研 → 内容起草 → SEO 优化 → 排期发布。

7. 企业知识库问答

将内部文档、Wiki、数据库接入Agent，构建专属企业专家问答系统。

8. "第二大脑"

随时给 AI 发消息让它记住任何信息，需要时搜索调用。配合 Next.js 仪表板可视化，打造真正的外部记忆。

9. IoT 设备控制

通过 Telegram 控制家里的空气净化器、远程关机电脑、调节智能灯光。

总结 & Q&A (2 分钟)

给所有人的一句话

Agent 不是一个神秘的黑箱。 它的能力来自精心设计的架构：一个可靠的思考引擎、一套丰富的工具、一个安全的权限体系、和一个让多个 Agent 协作的调度系统。理解了这些，你就能理解 AI Agent 能做什么、不能做什么，以及为什么有些 Agent 比其他的更好用。

三大收获

- Agent 的工作方式：**就像一个靠谱的员工——自主决策、边想边做、出了问题自己补救。技术上是 AsyncGenerator 状态机 + 流式执行 + 多级恢复策略。
- 五层架构：**
 - 内核（大脑和双手）→ 壳层（操作界面）→ 连接（对外通道）→ 治理（规则和管控）→ 规模（团队协作和记忆）
 - 由内而外，层层扩展，对应后续 7 条分享轨道
- 设计哲学：**模型驱动工具选择，框架控制执行编排。不需要人工编排复杂流程图——一个 while(true) 循环 + 统一的工具接口 + 完善的恢复机制就足够了。

后续轨道预告

日期	分组	分享主题	将深入哪些细节
4.10	1 - 对话引擎	Agent 运行引擎的工业级可靠性设计	query() 循环完整链路 · 流式响应 · 压缩策略
4.13	2 - 上下文与记忆	记忆系统和上下文工程研究框架	MEMORY.md · Session Memory · Extract · AutoDream

4.14	3 - 工具系统与权限	Agent的双手—工具和权限系统设计与应用	Tool 接口 · 48+ 工具 · 权限决策流
4.14	4 - 扩展与企业管控	Claude Code扩展生态设计与应用	Skills · Plugins · Hooks · 企业管控
4.15	5 - 多智能体与运行	Claude Code 多智能体编排：从源码到设计哲学	Fork / Teammate / Coordinator · 状态管理
4.16	6 - 终端产品与命令	窥探Claude Code 运行底座的设计哲学	React+Ink · 启动链 · 命令体系
4.17	7 - 集成与通信协议	Claude Code 集成与通信协议：Agent 如何连接世界	API · MCP · Bridge · 传输协议

本次概览是全系列的"总入口"——建立全局认知后，后续每条轨道都将深入某一层的技术细节。

附录：关键源文件速查表

组件	文件路径	说明
核心循环	<code>src/query.ts</code>	Agent 主循环 (~1730 行)
查询引擎	<code>src/QueryEngine.ts</code>	高层封装
工具定义	<code>src/Tool.ts</code>	Tool 类型系统
工具注册	<code>src/tools.ts</code>	工具发现和注册
工具执行	<code>src/services/tools/toolExecution.ts</code>	执行管道
工具编排	<code>src/services/tools/toolOrchestration.ts</code>	并发/串行策略
流式工具	<code>src/services/tools/StreamingToolExecutor.ts</code>	流式中执行工具
系统提示词	<code>src/constants/prompts.ts</code>	提示词组装

上下文管理	<code>src/context.ts</code>	系统/用户上下文
自动压缩	<code>src/services/compact/autoCompact.ts</code>	压缩策略
响应式压缩	<code>src/services/compact/reactiveCompact.ts</code>	溢出时被动触发
记忆系统	<code>src/memdir/memdir.ts</code>	持久化记忆
Agent 工具	<code>src/tools/AgentTool/AgentTool.tsx</code>	多 Agent 入口
协调者模式	<code>src/coordinator/coordinatorMode.ts</code>	多 Agent 编排
Swarm	<code>src/utils/swarm/</code>	团队协作基础设施
技能系统	<code>src/skills/loadSkillsDir.ts</code>	技能加载
插件系统	<code>src/plugins/builtinPlugins.ts</code>	插件框架
权限系统	<code>src/utils/permissions/permissions.ts</code>	权限检查
权限类型	<code>src/types/permissions.ts</code>	权限模式定义
API 客户端	<code>src/services/api/claude.ts</code>	Anthropic API
MCP 客户端	<code>src/services/mcp/client.ts</code>	MCP 协议
设置管理	<code>src/utils/settings/setting.ts</code>	配置系统
企业策略	<code>src/services/policyLimits/index.ts</code>	组织限制
启动入口	<code>src/entrypoints/cli.tsx</code>	CLI 入口
主 TUI	<code>src/main.tsx</code>	主逻辑
REPL 界面	<code>src/screens/REPL.tsx</code>	交互界面

Bridge	src/bridge/bridgeMain.ts	远程桥接
遥测	src/utils/telemetry/instrumentation.ts	OpenTelemetry
分析	src/services/analytics/index.ts	事件分析